

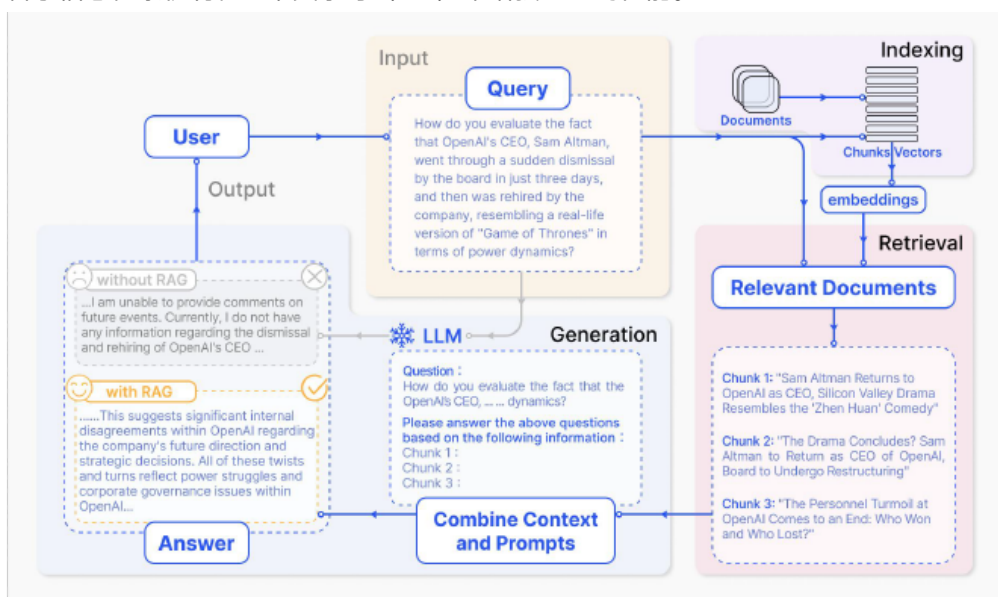
RAG技术汇报

1. 背景

- LLM的缺点
 - 幻觉（会生成无关的内容）
 - 信息更新不及时（例如出现一些新的学科知识、出现新的论文）
 - 对于专业领域的知识缺乏深度
 - 推理过程脆弱（依赖于prompt的质量）
- 实际的需求
 - 领域知识回答的准确度
 - 数据需要频繁更新（新题目）
 - 生成的内容可以溯源（提供可解释性）
 - 成本控制（随数据的迭代进行更新成本高）
 - 隐私保护

2. 什么是RAG?

- 定义：RAG是一个通过整合外部知识库来增强LLM的范式。它采用了一种协同方法，结合了信息检索机制和上下文学习（ICL）来增强LLM的性能。



- - Indexing: 将已有的知识型数据建立成知识库，需要建立索引
 - Retrieval: 通过检索知识库中与用户query相关的内容来获取专业知识
 - Generation: 将检索得到的内容与原始的query进行融合后送入模型用于生成回答

3. RAG框架

- Naive RAG→Advanced RAG→Modular RAG

Naive RAG

Step1 Indexing

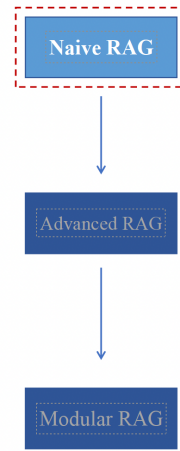
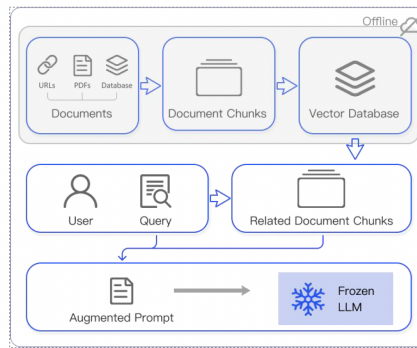
1. 将文档分成多个大小相同的块。
2. 使用编码器对这些划分的块进行编码，生成embedding。
3. 将得到的embedding存储到向量数据库中。

Step2 Retrieval

通过相似性检索与问题最相关的k个块的内容。

Step3 Generation

拼接原始的提问&检索得到的k个文本的内容作为模型的输入。

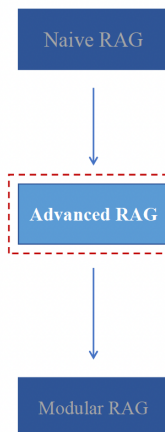
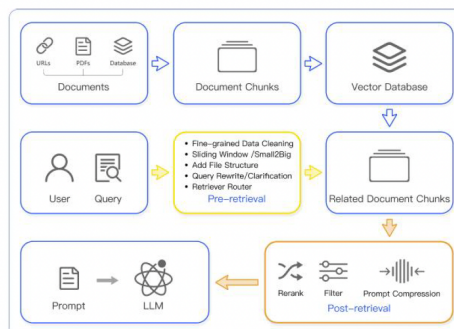


- 索引 (Indexing)
 - 数据清洗&提取
 - 分块&embedding
 - 建立索引来存储文本块&向量嵌入组成的键值对
- 检索 (Retrieval)
 - query向量embedding
 - 计算query向量嵌入表示与语料库中所有向量块之间的相似度分数
 - 取出相似度最高的top k个块嵌入表示对应的知识
- 生成 (Generation)
 - 将原始query&检索得到的块知识融合得到新的prompt
 - 将新的prompt输入LLM得到输出
- 缺点:
 - 检索质量低会导致精度低，且仍然存在幻觉问题
 - 知识库构建方法太粗糙
 - query&知识库中知识相关程度低（需要重写 | 分类）
 - 回复生成的数据存在冗余、重复率高的问题
 - 缺少索引后处理，导致检索的知识重复率高（需要重排 | prompt压缩）

Advanced RAG

Index Optimization → Pre-Retrieval Process → Retrieval → Post-Retrieval Process → Generation

- **Optimizing Data Indexing:** 滑动窗口、细粒度的划分、添加元信息、长文本建立索引
- **Pre-Retrieval Process:** 索引链路、总结、重写、置信度
- **Post-Retrieval Process:** 重排、过滤、prompt压缩



- Data Indexing
 - 细粒度划分：分块、词粒度、句子粒度、知识图谱
 - Token: [1911.00172 \(arxiv.org\)](https://arxiv.org/abs/1911.00172)
 - 解说: [香依读 | 用上文K最近邻特征表示增强语言模型 - 知乎 \(zhihu.com\)](https://www.zhihu.com/question/35484444)

- Phrase: [2212.01349 \(arxiv.org\)](https://arxiv.org/abs/2212.01349)
 - 解说: [Meta发布首个「非参数化」掩码语言模型NPM：吊打500倍参数数量的GPT-3 – 知乎 \(zhihu.com\)](https://zh.wikipedia.org/wiki/Meta发布首个「非参数化」掩码语言模型NPM：吊打500倍参数数量的GPT-3)
 - code: [code](#)
- Chunk: [2302.00083 \(arxiv.org\)](https://arxiv.org/abs/2302.00083)
 - 解说: [TACL | In-Context Retrieval-Augmented Language Models – 知乎 \(zhihu.com\)](https://zh.wikipedia.org/wiki/TACL|In-Context Retrieval-Augmented Language Models)
 - code: <https://github.com/AI21Labs/in-context-ralm>
 - 分块的策略: 字符分割 (基于静态字符)、递归字符文本分割、特定文档分割 (PDF、Word、Markdown)、语义分割 (基于嵌入遍历的分块)、LLM 分割
 - 语义分割: 利用成分句法分析、关键词提取
- 添加元信息: 标题、页码、时间戳、数据标签等 (构建多级索引 | 检索时分流)
- 长文本建立索引
 - Small-2-Big: 句子级别进行嵌入 (Small), 检索后会拉取该句子上面几行 + 下面几行的数据 (Big) 作为检索到的内容
 - 核心: {key: 一句话; value: 前面几句话+key+后面几句话}
 - <https://towardsdatascience.com/advanced-rag-01-small-to-big-retrieval-172181b396d4>
 - Summary: 使用LLM | 摘要抽取模型提取的总结/摘要作为该文本的索引块
 - Sliding window: 使用滑动窗口来切分文本, 保证索引库中包含文档的所有内容
 - 上下文信息依赖于窗口的大小&步长
 - 添加路由功能: 通过建立多级索引搭配路由技术可以提升处理效率, 同时保留足够长的文本信息
 - <https://github.com/aurelio-labs/semantic-router>
 - 对文本生成假设性问题: 让LLM为每个块生成一个假设性问题, 将这些假设性问题转为向量, 与query向量进行匹配, 并将检索的真实的块与真实query进行融合后输入模型
 - 这种方法由于查询和假设性问题之间的语义相似性更高, 相较于实际数据块, 可以提升搜索质量。
 - FLARE (见后面): [2305.06983 \(arxiv.org\)](https://arxiv.org/abs/2305.06983)
 - HyDE: <https://arxiv.org/pdf/2212.10496>

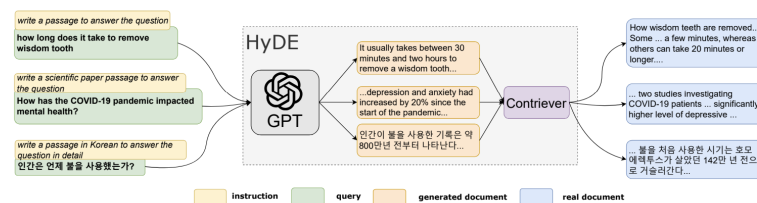


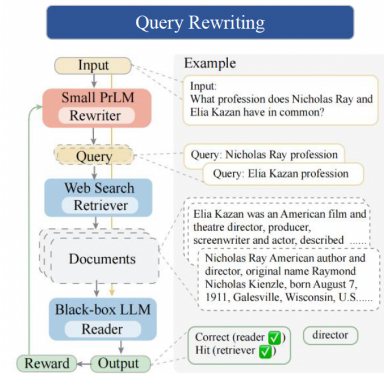
Figure 1: An illustration of the HyDE model. Documents snippets are shown. HyDE serves all types of queries without changing the underlying GPT-3 and Contriever/mContriever models.

- 使用LLM生成回答, 用回答去检索对应的知识块
- Pre-Retrieval Process
 - Query重写
 - 原因: 问题&答案并不总是高语义相关的, 可以通过LLM来调整query
 - 方法: 基于文本抽取模型 | 基于LLM
 - 基于文本摘要模型可以使用开源模型

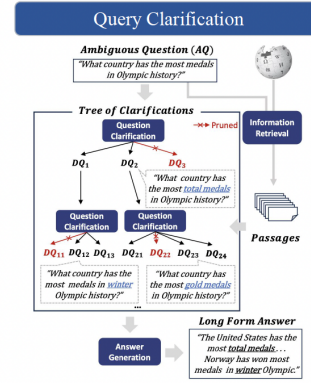
■ 基于LLM进行重写

► Techniques for Better RAG — Query Optimization

Questions and answers do not always possess high semantic similarity; adjusting the Query can yield better retrieval results.



Rewrite-Retrieve-Read [Ma et al., 2023]



Tree of Clarifications (TOC) [Kim et al., 2023]

- RRR: <http://arxiv.org/pdf/2305.14283>

■ Query扩展

- 原因：单一query可能无法在知识库中检索到符合答案的内容
- 方法：使用LLM生成多个query的扩展，并执行查询，主要关注多样性和覆盖度

- code: <https://github.com/xbmxb/RAG-query-rewriting>

- TOC: [2310.14696 \(arxiv.org\)](https://arxiv.org/pdf/2310.14696)

- 生成多个子Query，分别进行检索，将大问题拆分成小问题

- code: [TOC](#)

□ 索引链路（路由）

- 原因：单一模型可能存在局限性，不同场景需要使用不同的RAG系统
- 方法：将不同的Query“路由”到不同的RAG pipeline中

■ Post-Retrieval Process

□ 重排

- 原因：检索的内容过多，存在重复和冗余，同时需要将最相关的内容放在检索的首位
- 核心：根据Query与检索内容相似度 | 业务指标来对检索结果进行重排
- 方法：基于重排模型 | 基于LLM

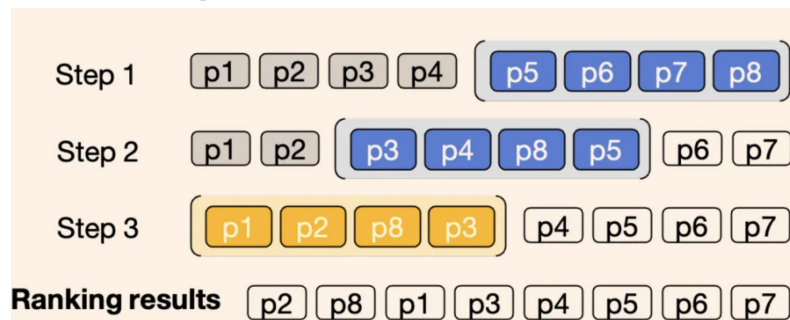
- 基于重排模型可以输出文档与查询之间的相关性

- 基于LLM可以更全面捕捉语义信息

- 提示LLM进行重排（使用滑动窗口）

- RankGPT: <http://arxiv.org/pdf/2304.09542>

- code: https://gitcode.com/sunnweiwei/RankGPT/overview?utm_source=artical_gitcode

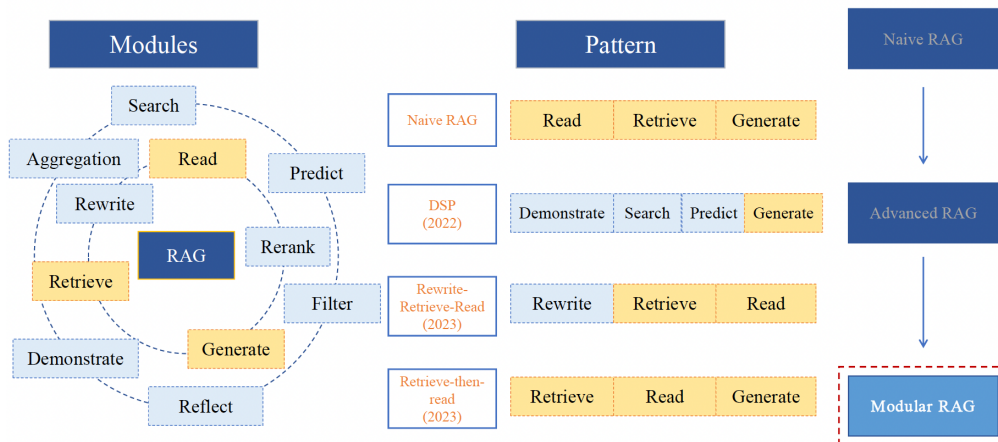


- 训练中使用LLM进行增强
- 将LLM与重排任务一起微调

Embedding	WithoutReranker		bge-reranker-base		bge-reranker-large		Cohere-Reranker	
	Hit Rate	MRR	Hit Rate	MRR	Hit Rate	MRR	Hit Rate	MRR
OpenAI	0.876404	0.718165	0.91573	0.832584	0.910112	0.855805	0.926966	0.86573
bge-large	0.752809	0.597191	0.859551	0.805243	0.865169	0.816011	0.876404	0.822753
llm-embedder	0.814607	0.587266	0.870787	0.80309	0.876404	0.824625	0.882022	0.830243
Cohere-v2	0.780899	0.570506	0.876404	0.798127	0.876404	0.825281	0.876404	0.815543
Cohere-v3	0.825843	0.624532	0.882022	0.806086	0.882022	0.834644	0.88764	0.836049
Voyage	0.831461	0.68736	0.926966	0.837172	0.91573	0.858614	0.91573	0.851217
JinaAI-Small	0.831461	0.614045	0.91573	0.843071	0.926966	0.857303	0.926966	0.868633
JinaAI-Base	0.848315	0.68221	0.938202	0.846348	0.938202	0.868539	0.932584	0.873689
Google-PaLM	0.865169	0.719476	0.910112	0.833708	0.910112	0.85309	0.910112	0.855712

- 实验结论：
 - 无论使用何种嵌入模型，加入重排都可以提升效果
 - 使用开源模型需要考虑与embedding模型的搭配
- Cohere效果最好，提供api，需要付费
- bge-reranker
 - 优点：开源、效果与Cohere效果最接近
 - 开箱即用：<https://github.com/FlagOpen/FlagEmbedding>
- 实验：<https://zhuanlan.zhihu.com/p/673119562>
- 信息压缩（检索内容过长会超过文本限制长度，也可能会引入噪音）
 - 原因：检索文档中噪音数据对RAG性能有负面影响
 - 核心：压缩不相关的上下文内容，突出关键段落
 - 方法：基于文本抽取模型 | 基于LLM
 - 基于压缩模型可以直接使用开源的模型来进行抽取
 - 基于LLM来进行重写

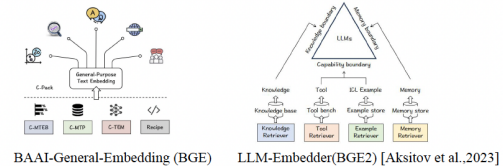
► Modular RAG



- 集成前面两种方法，并且提供了更多的模块用于提高RAG的能力
 - 搜索模块
 - 可以在额外的语料库上进行检索，例如使用搜索引擎
 - 记忆模块
 - 将LLM的输出用于改进知识库（双重query）
 - 融合
 - 通过多查询方法来缓解单一query的局限性
 - 路由
 - 通过构建不同数据来源/模态信息的知识库，进行自由组合
 - 预测
 - 利用LLM生成的内容+query组后用于检索
- 优点：知识库比较全面，检索更准确
- 缺点：延迟高，实现的复杂度高

4. RAG关键问题

- 如何选择索引方法
 - 稀疏索引方法&稠密索引方法
 - 稀疏索引方法：BM25、TF-IDF
 - 稠密索引方法：embedding模型
 - 基于BERT：ColBERT
 - 基于多任务微调：BGE&Baichuan-Text-Embedding
 - 基于云API的模型：OpenAI-Ada-002&Cohere Embedding
 - 基于编码框架：Dragon+
- embedding模型
 - 如何选取embedding模型



- 索引内容的性质（中英文等）
- 不同embedding模型对块大小的适配度
 - sentence-transformer在单个句子上表现好
 - text-embedding-ada在256-512tokens的块中表现好
- 用户query的预期长度和复杂性
- 检索后的应用场景
 - 语义搜索 or Q&A
- 如何利用开源的embedding模型
 - 根据评测结果直接使用开源embedding模型（Retrieval:使用ndcg作为评价指标）

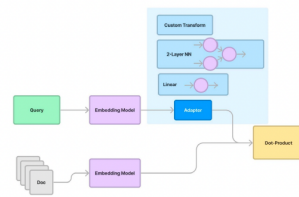
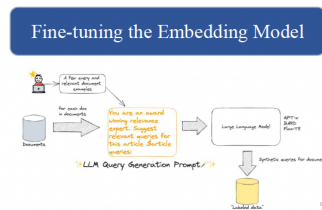
Rank	Model	Model Size (Million Parameters)	Memory Usage (GB, fp32)	Average	CmedqaRetrieval	CovidRetrieval	DuRetrieval
1	360Zhinao-search	163	0.61	75.06	46.73	85.02	87.57
2	AGE_Hybrid			74.97	47.26	83.66	89.28
3	OpenSearch-text-hybrid			74.41	46.56	84.03	87.85
4	piccolo-large-zh-v2			74.36	47.58	86.78	89.14
5	PEG	163	0.61	73.78	44.42	82.56	87.67
6	stella-large-zh-v3-1792d	325	1.21	73.6	46.87	82.44	87.13
7	stella-mrl-large-zh-v3.5-1792d	326	1.21	73.52	46.61	81.87	86.8
8	xiaobu-embedding	326	1.21	73.41	44.22	87.69	86.45
9	alime-embedding-large-zh	326	1.21	73.3	43.93	88.14	86.23
10	Baichuan-text-embedding			73.12	47.64	86.86	88.43
11	acge_text_embedding	326	1.21	72.93	46.78	81.56	86.55

- Reranker:使用map作为评价指标

Rank	Model	Model Size (Million Parameters)	Memory Usage (GB, fp32)	Average	CMedQAv1	CMedQAv2	MMarcoReranking	T2Reranking
1	360Zhinao-1.88-Reranking	941	3.51	70.13	86.75	87.92	37.29	68.55
2	piccolo-large-zh-v2			70	89.31	90.14	33.39	67.15
3	Baichuan-text-embedding			69.67	88.06	88.46	34.3	67.85
4	360Zhinao-search	163	0.61	68.92	87	88.48	32.41	67.8
5	AGE_Hybrid			68.72	89.37	89.27	29.64	66.61
6	stella-mrl-large-zh-v3.5-1792d	326	1.21	68.45	89.33	89.18	28.85	66.43
7	PEG	163	0.61	68.41	84.09	86.56	33.55	69.43
8	OpenSearch-text-hybrid			68.27	88.99	89.6	28.12	66.38
9	stella-large-zh-v3-1792d	325	1.21	68.26	88.99	89.6	28.09	66.38
10	LYun-large-zh			68.08	88.57	88.9	28.24	66.62

- <https://huggingface.co/spaces/mteb/leaderboard>

- 微调embedding模型



Fine-tuning According to Domain-Specific Repositories and Downstream Tasks

Fine-tuning the Adapter Module to Align the Embedding Model with the Retrieval Repository

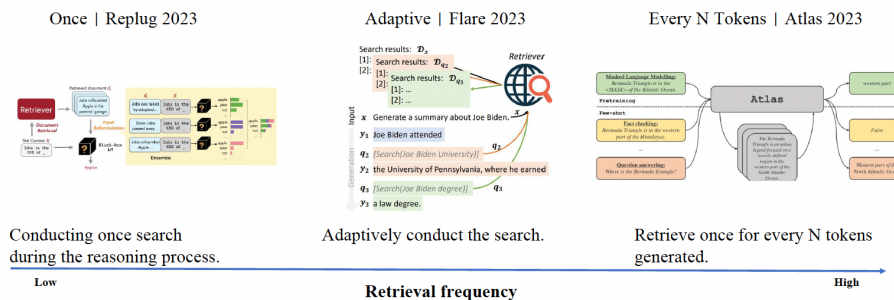
- 领域知识微调
 - 微调数据集三要素：query、语料库、与query相关文档（标签）
- 下游任务微调
 - 结合LLM来进行微调
- 检索
 - 检索内容的质量
 - 重写query
 - 嵌入转换（对齐query与知识库）
 - 检索的时机、次数

► Key issue of RAG — When to retrieve

High efficiency, but low relevance of the retrieved documents

Balancing efficiency and information might not yield the optimal solution

A large amount of information with low efficiency and redundant information.



- 检索一次，也即在推理层面使用检索的信息
 - REPLUG: [2301.12652 \(arxiv.org\)](https://arxiv.org/abs/2301.12652)
 - 将LLM看作黑盒，将检索的多个内容拼接query输入LLM

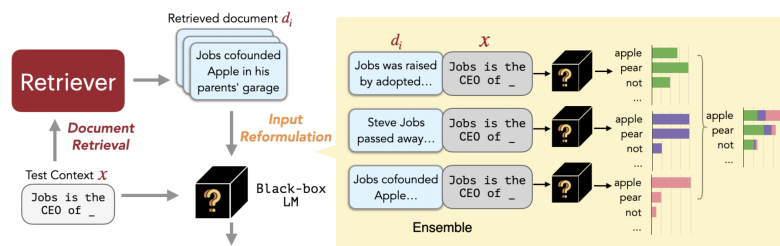


Figure 2. REPLUG at inference (§3). Given an input context, REPLUG first retrieves a small set of relevant documents from an external corpus using a retriever (§3.1 Document Retrieval). Then it prepends each document separately to the input context and ensembles output probabilities from different passes (§3.2 Input Reformulation).

- 解说: [REPLUG: 检索增强的黑盒子语言模型 – 知乎 \(zhihu.com\)](https://zh.wikipedia.org/wiki/REPLUG)
- 自适应，也即使用模型来判断检索信息的好坏
 - FLARE: [2305.06983 \(arxiv.org\)](https://arxiv.org/abs/2305.06983)
 - 利用多个LLM来实现自适应地提升检索的质量
 - 使用一个LLM生成答案，一个LLM生成假设性问题用于检索，一个检索器
 - 当LLM生成的回答置信度高时，则直接使用

- 当LLM生成的回答置信度低的时候（提炼关键词重新检索、使用LLM改写query，提出更可能被检索的问题）

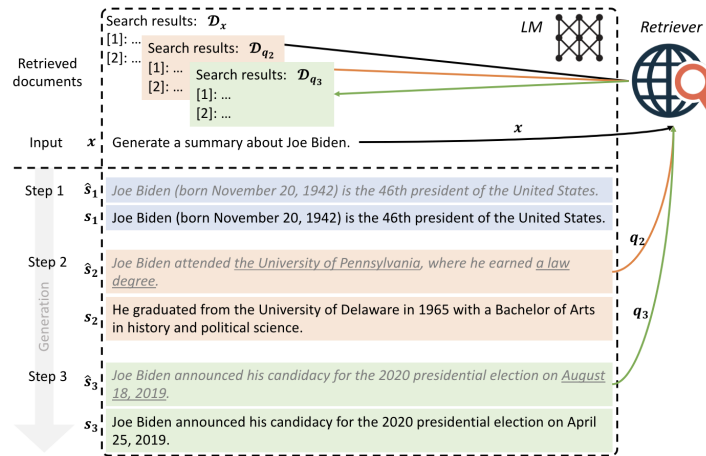


Figure 1: An illustration of forward-looking active retrieval augmented generation (FLARE). Starting with the user input x and initial retrieval results $\mathcal{D}_x$, FLARE iteratively generates a temporary next sentence (shown in *gray italic*) and check whether it contains low-probability tokens (indicated with underline). If so (step 2 and 3), the system retrieves relevant documents and regenerates the sentence.

- code: [FLARE](#)
- 解说: [使用主动检索增强生成 FLARE 实现更优越的 RAG – 知乎 \(zhihu.com\)](#)
- 每N个token
 - Atlas: [2208.03299 \(arxiv.org\)](#)
 - 每N个token作为一个索引块，使用前N/2个token作为query，后N/2个token作为输出，在索引阶段只使用前N/2个token来检索相关的文档

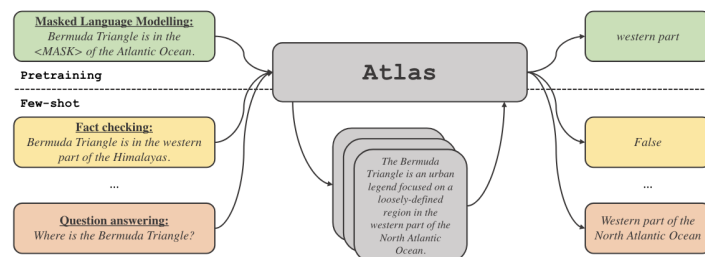
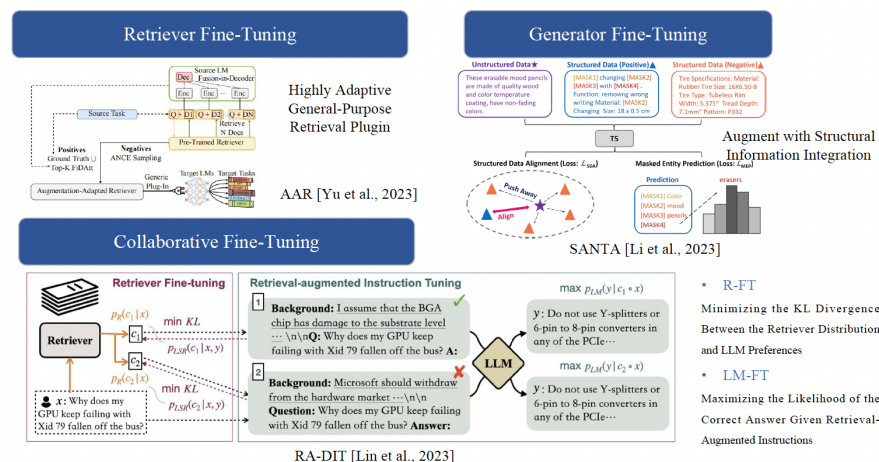


Figure 1: We introduce ATLAS, a retrieval-augmented language model that exhibits strong few-shot performance on knowledge tasks, and uses retrieval during both pre-training and fine-tuning.

- code: [code](#)
- 解说: [Meta推出110亿参数模型Atlas，用64个训练实现42.4%的准确率 – 知乎 \(zhihu.com\)](#)
- 对齐检索器与LLM
 - 微调检索器



■ RA-DIT: [2310.01352 \(arxiv.org\)](https://arxiv.org/abs/2310.01352)

□ 两阶段进行微调

- 第一阶段：根据索引得到的知识来微调LLM
- 第二阶段：根据LLM输出的得分来优化检索器（KL散度）

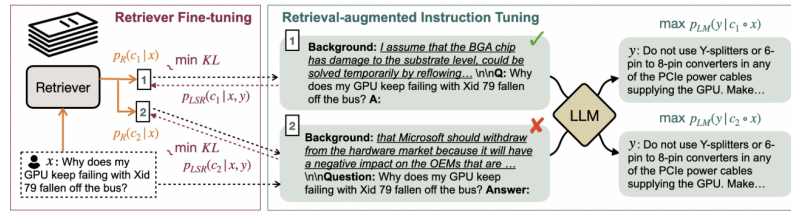


Figure 1: The RA-DIT approach separately fine-tunes the LLM and the retriever. For a given example, the LM-ft component updates the LLM to maximize the likelihood of the correct answer given the retrieval-augmented instructions (§2.3); the R-ft component updates the retriever to minimize the KL-Divergence between the retriever score distribution and the LLM preference (§2.4)

□

□ 解说: [RA-DIT- 知乎 \(zhihu.com\)](https://www.zhihu.com/question/35484444/answer/15484444)

■ RAFT: <https://arxiv.org/pdf/2403.10131>

□ 使用LLM剔除对回答问题没有帮助的通过检索获得的知识（文档）

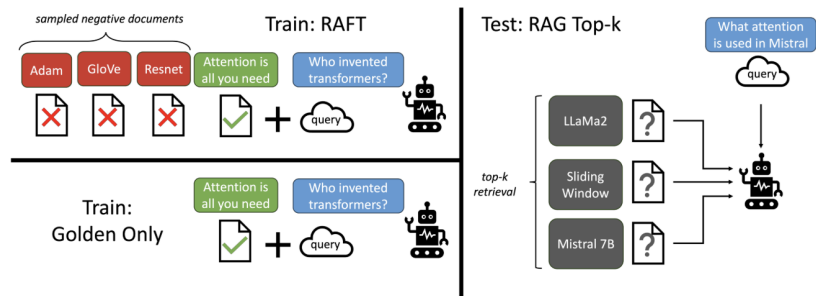


Figure 2: **Overview of our RAFT method.** The top-left figure depicts our approach of adapting LLMs to reading solution from a set of positive and negative documents in contrast to standard RAG setup where models are trained based on the retriever outputs, which is a mixture of both memorization and reading. At test time, all methods follow the standard RAG setting, provided with a top-k retrieved documents in the context.

□

□ code: <https://github.com/ShishirPatil/gorilla>

■ AAR: [2305.17331 \(arxiv.org\)](https://arxiv.org/abs/2305.17331)

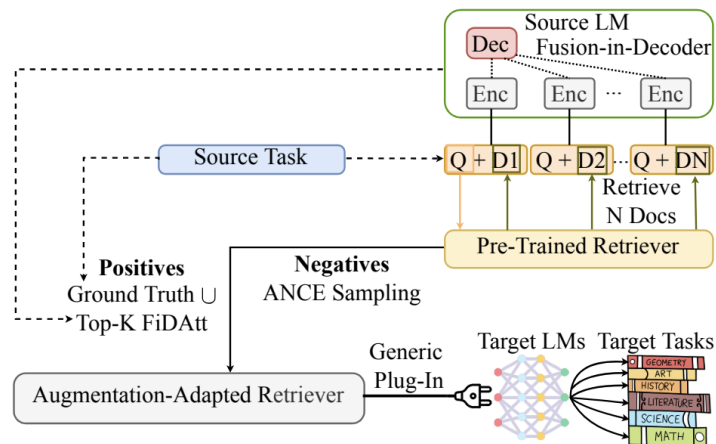


Figure 2: Illustration of augmentation-adapted retriever.

□

- ANCE: <https://arxiv.org/pdf/2007.00808>
- 针对索引器&目标LLM不能进行协同微调的情况,
 - 在source LLM上进行微调后用于target LLM

□ code: [code](#)

■ SANTA: [2305.19912 \(arxiv.org\)](https://arxiv.org/abs/2305.19912)

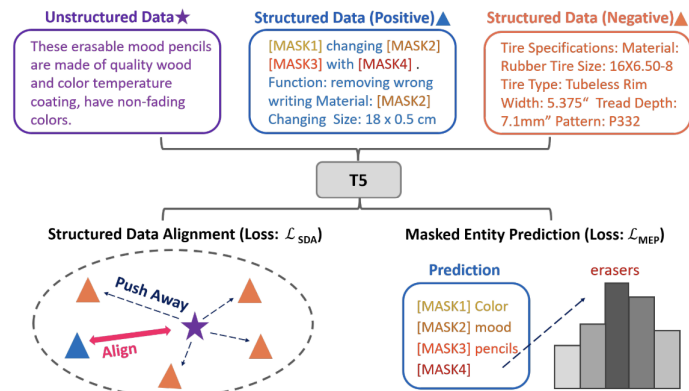


Figure 2: The Structure-Aware Pretraining Methods of SANTA. We use both Structured Data Alignment (SDA) and Masked Entity Prediction (MEP) methods for pretraining.

- 考虑到输入数据的多样性（结构化数据、非结构化数据，负样本）
 - code: <https://github.com/OpenMatch/OpenMatch>
- 如何利用检索的信息提高LLM的能力（生成）
 - 加入到预训练侧
 - InstructRetro: <https://arxiv.org/pdf/2310.07713>
 - 预训练LLM→ 加入检索器继续预训练LLM→指令微调

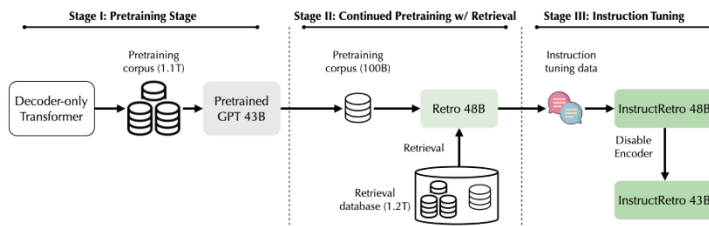


Figure 1: Training pipeline for InstructRetro 48B and InstructRetro 43B.

- code: [code](#)
- 加入到微调侧
 - Self-RAG: [2310.11511 \(arxiv.org\)](https://arxiv.org/abs/2310.11511)
 - 提升语言模型生成质量与准确性
 - 检索（自适应，检索标记）→自我反思（反思标记）→批评生成（批评标记）
 - 检索标记：是否需要使用检索器进行检索
 - 反思标记：评估检索知识的相关性、支持度以及完整性
 - 批评标记：评估生成回答的质量

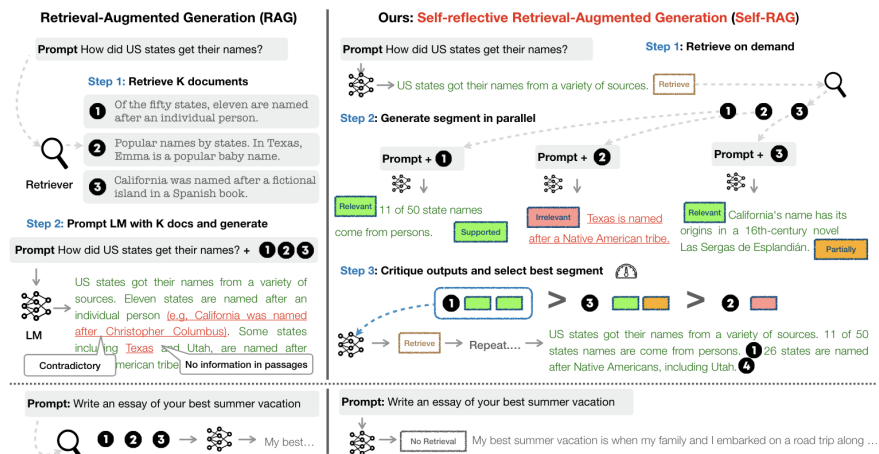


Figure 1: Overview of SELF-RAG. SELF-RAG learns to retrieve, critique, and generate text passages to enhance overall generation quality, factuality, and verifiability.

- code: [self-rag](#)
- 解说: [SELF-RAG: 自我反思检索增强生成](#)

■ 加入到推理侧

□ GenRead: [2209.10063 \(arxiv.org\)](https://arxiv.org/abs/2209.10063)

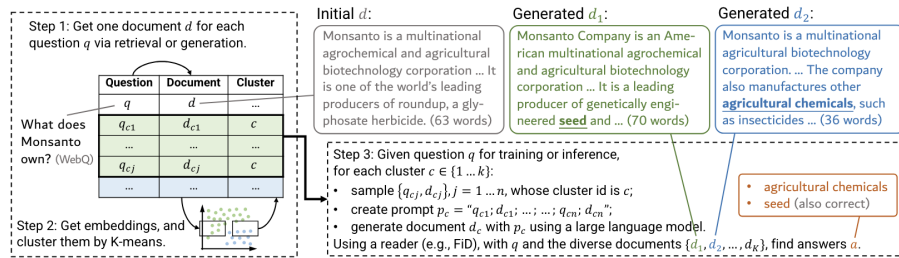
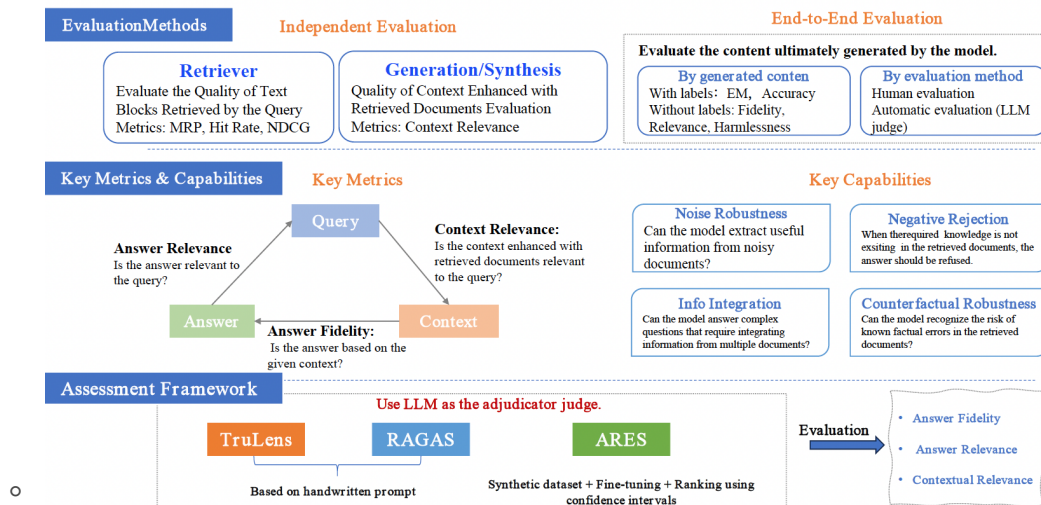


Figure 1: An overall framework of clustering-based prompting method. It leverages distinct question-document pairs sampled from each embedding cluster as in-context demonstrations to prompt a large language model to generate diverse documents, then read the documents to predict an answer.

-
- 将query&文档拼接得到一个样本对，获取embedding之后使用k-means进行聚类，然后将聚类的结果拼接成prompt送入LLM生成多样性的结果
- code: <https://github.com/wyu97/GenRead>

5. 评估

► How to Evaluate the Effectiveness of RAG



■ 评估指标

□ HR、NDCG、MRR

■ 评估框架

Evaluation Framework	Evaluation Targets	Evaluation Aspects	Quantitative Metrics
RGB [†]	Retrieval Quality Generation Quality	Noise Robustness	Accuracy
		Negative Rejection	EM
		Information Integration	Accuracy
		Counterfactual Robustness	Accuracy
RECALL [†]	Generation Quality	Counterfactual Robustness	R-Rate (Reappearance Rate)
RAGAS [‡]	Retrieval Quality Generation Quality	Context Relevance	*
		Faithfulness	*
		Answer Relevance	Cosine Similarity
ARES [‡]	Retrieval Quality Generation Quality	Context Relevance	Accuracy
		Faithfulness	Accuracy
		Answer Relevance	Accuracy
TruLens [‡]	Retrieval Quality Generation Quality	Context Relevance	*
		Faithfulness	*
		Answer Relevance	*
CRUD [†]	Retrieval Quality Generation Quality	Creative Generation	BLEU
		Knowledge-intensive QA	ROUGE-L
		Error Correction	BertScore
		Summarization	RAGQuestEval

□ RAGAS: <https://arxiv.org/pdf/2309.15217>

- 评估检索质量，使用上下文相关性、忠实度和回答相关性等指标
- code: <https://github.com/explodinggradients/ragas>

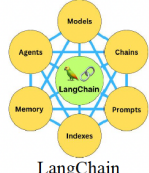
- 解说: <https://mp.weixin.qq.com/s/nbO71eti8FI00K5OiKzBQ>
- ▣ Metb: <https://arxiv.org/pdf/2210.07316>
 - 评测榜单: <https://huggingface.co/spaces/mteb/leaderboard>
 - 使用方法: <https://cloud.tencent.com/developer/article/2391350>
 - code: <https://github.com/embeddings-benchmark/mteb>
- ▣ RGB: <https://arxiv.org/pdf/2309.01431.pdf>
 - 评估检索质量, 使用准确性 (EM)、召回率 (Recall)、精确度 (Precision)、R-Rate (重新出现率)、余弦相似度 (Cosine Similarity)、Hit Rate和MRR (Mean Reciprocal Rank) 等指标
 - 解说: <https://zhuanlan.zhihu.com/p/664265300>
- ▣ RECALL: <http://arxiv.org/pdf/2311.08147>
 - 评估生成质量, 使用准确性 (Accuracy)、R-Rate、Hit Rate和MRR等指标
- ▣ ARES: <https://arxiv.org/pdf/2311.09476>
 - 评估检索质量, 使用上下文相关性、忠实度和回答相关性等指标
 - code: <https://github.com/stanford-futuredata/ARES>
- ▣ TruLens: https://www.trulens.org/trulens_eval/
 - 评估检索质量, 使用上下文相关性、忠实度和回答相关性等指标
- ▣ CRUD: <https://arxiv.org/abs/2401.17043>
 - 涵盖 RAG 应用的增删改查 (CRUD) 四种不同场景
 - Create, Read, Update, and Delete
 - code: https://github.com/IAAR-Shanghai/CRUD_RAG

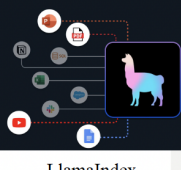
6. 开源

- 相关框架

► Existing Tech Stack for RAG

Name	Pros	Cons
LangChain	Modular, full-featured	Inconsistent behavior, API conceals details, complexity and low flexibility.
LlamaIndex	Focus on RAG	Requires combination use, low customization.
FlowiseAI	Easy to get started, visualized workflows.	Does not support complex scenarios.
AutoGen	Adapts to multi-agent scenarios.	Low efficiency, requires multiple rounds of dialogue.




- LangChain
 - ▣ 定义: 一个使用LLM开发应用的框架
 - ▣ 优点: 提供了标准的内存接口和内存实现, 支持自定义大模型的封装
 - ▣ 缺点: 评估生成模型的性能比较困难
 - ▣ 开箱即用: <https://github.com/langchain-ai/langchain>
 - ▣ 使用文档: https://python.langchain.com/docs/get_started/introduction
 - ▣ 上手指南: <https://zhuanlan.zhihu.com/p/668650637>
- LlamaIndex
 - ▣ 定义: 一个专用于构建RAG系统的框架
 - ▣ 优点: 专注于索引和检索, 易于上手

- 缺点：生态比LangChain小，限制了一些高级功能和优化策略的获取
- 开箱即用：https://gitcode.com/run-llama/llama_index/overview?utm_source=csdn_github_accelerator
- 使用文档：<https://docs.llamaindex.ai/en/stable/>
- 上手指南：https://blog.csdn.net/qq_41185868/article/details/134257300
- Flowise：<https://gitcode.com/FlowiseAI/Flowise>
- AutoGen：<https://github.com/microsoft/autogen>
- 相关工具
 - QAnything
 - 优点：支持任何格式文件或者数据库本地知识库，支持跨语文问答、粗排、精排
 - <https://github.com/netease-youdao/QAnything>
 - 在线试用地址：<https://qanything.ai/#/>
 - 使用方法：<https://zhuanlan.zhihu.com/p/676770191>
 - FastGPT
 - 优点：提供了数据处理、模型调用等能力，可以使用Flow进行工作流编排，支持接入企业微信等
 - <https://github.com/labring/FastGPT>
 - 使用方法：<https://zhuanlan.zhihu.com/p/674206944>
 - RAGFlow：
 - 优点：对文档类型数据比较友好，提供了多种不同格式的文件解析方法
 - <https://github.com/infiniflow/ragflow>
 - 使用方法：<https://zhuanlan.zhihu.com/p/692901955>

7. 参考资料

- RAG-survey：<https://arxiv.org/abs/2312.10997>
 - 资源：<https://github.com/Tongji-KGLLM/RAG-Survey>
 - 解读：<https://zhuanlan.zhihu.com/p/683171274>
- Langchain：https://zhuanlan.zhihu.com/p/656646499?utm_id=0